Test Automation Proof of Architecture — 4-Week Plan

Context: ~20 desktop/web/POS/Unix applications, ~1,500 existing manual test scenarios.
Goal: Fact-based, POC-backed recommendation of a test automation tool (or toolchain) with a realistic automation coverage target and adoption roadmap.

Guiding principles

- Don't evaluate tools in the abstract — evaluate them against *this* client's estate.
- Select POC applications that represent the hardest 20% of the estate (POS with peripherals, legacy desktop, terminal apps), not the easiest.
- Measure everything during the POC: script creation time, execution time, flakiness, maintenance effort after an app change.
- Present coverage as a fact-based number ("X% of the 1,500 scenarios are automatable, here is the breakdown"), not a promise of 100%.

Week 1 — Discovery & Estate Analysis

Day 1 — Kickoff & access

- Kickoff meeting: confirm scope, success criteria, stakeholders, decision-makers.
- Request access: applications, test environments, test management tool (Jira/ALM/Xray/Zephyr), CI/CD, source control.
- Agree on the definition of "done" for the POA (deliverables: assessment report, POC results, recommendation, roadmap).

Day 2 — Application inventory

- Build an application landscape matrix for all ~20 apps: name, type (web/desktop/POS/terminal), technology (e.g., .NET WinForms/WPF, Java Swing, Electron, Angular/React, mainframe/curses, embedded POS), version, hosting, release cadence, criticality.
- Identify integrations between apps (end-to-end flows crossing app boundaries — these drive tool choice heavily).

Day 3 — Test scenario analysis

- Export the ~1,500 scenarios; classify by: application, type

(functional/regression/smoke/E2E), execution frequency, data dependency, environment dependency, manual-only constraints (hardware, printers, card readers, physical scanners).
- Tag each scenario: *automatable now / automatable with effort / not automatable* — this becomes your coverage baseline.

Day 4 — Stakeholder interviews & constraints

- Interview QA leads, dev leads, ops: current pain points, past automation attempts and why they failed, team skill profile (coders vs. non-coders), budget appetite (commercial vs. open source).
- Capture non-functional constraints: security policies, on-prem vs. cloud, licensing procurement rules, CI/CD stack, preferred languages.

Day 5 — Evaluation framework & tool longlist

- Define weighted evaluation criteria (example): technology coverage (25%), script creation effort (15%), maintainability/object recognition robustness (15%), CI/CD & reporting integration (10%), skill fit (10%), licensing/TCO (15%), vendor support & roadmap (10%).
- Build the longlist based on estate facts. Typical candidates for a mixed estate:
  - **Commercial, broad coverage:** Tricentis Tosca, OpenText UFT One, Eggplant (image/AI-based — strong for POS), Ranorex, TestComplete.
  - **Open source stack:** Robot Framework (with SeleniumLibrary/Browser, FlaUI/WinAppDriver for desktop, SSHLibrary for Unix), Playwright/Selenium for web, Appium/WinAppDriver, Sikuli/OpenCV for image-based fallback.
  - **Hybrid:** open-source web + commercial tool only for POS/legacy desktop.
- Get client sign-off on criteria and weights **before** the POC — this protects the objectivity of the final recommendation.

Week 2 — Shortlisting & POC Setup

Day 6 — Shortlist & POC scope

- Score the longlist on paper against the criteria; shortlist 2–3 tools/stacks for hands-on POC.
- Select 3–4 representative applications: one modern web, one legacy desktop, one POS, one Unix/terminal.
- Select 10–15 POC scenarios covering: simple flow, data-driven flow, cross-application E2E flow, one "hard" case (peripheral simulation, dynamic controls, terminal screen

scraping).

Day 7 — Environment & licenses

- Obtain trial licenses / install open-source stacks; provision POC machines/VMs.
- Verify each tool can actually *see* the apps (object spy / locator inspection on each app type) — this alone eliminates weak candidates fast.

Day 8 — POC design

- Define the POC scorecard: metrics to capture per tool per scenario (time to automate, lines/steps, stability over N runs, object recognition quality, debugging experience).
- Design a small framework skeleton per tool: page objects/modules, test data handling, reporting.

Day 9 — Build: Tool 1

- Automate the POC scenarios in the first shortlisted tool. Log effort and blockers meticulously.

Day 10 — Build: Tool 1 (continued) + checkpoint

- Finish Tool 1 scenarios, run 10+ times to measure stability.
- Weekly checkpoint with client: findings so far, estate analysis results, confirmed automatable coverage estimate.

Week 3 — POC Execution & Comparison

Day 11–12 — Build: Tool 2

- Automate the same scenarios in the second tool. Same metrics, same rigor.
- Pay attention to where each tool struggles (dynamic IDs, custom controls, terminal emulation, image matching under different resolutions).

Day 13 — Build: Tool 3 (or hybrid stack)

- Automate the same scenarios in the third option (often the hybrid: e.g., Playwright + Robot Framework + image-based tool for POS).

Day 14 — Integration & operability tests

- For each tool: integrate with the client's CI/CD (Jenkins/Azure DevOps/GitLab),

trigger a run, publish reports.
- Test version control workflow, parallel execution, scheduling, and test-management integration (results back to Jira/ALM).

Day 15 — Maintainability & resilience test

- Simulate change: modify a screen/locator in one app and measure fix effort per tool.
- Run full POC suites 20+ times; record flakiness rate per tool.
- Capture licensing quotes / TCO inputs (3-year view: licenses, infra, training, ramp-up effort).

Week 4 — Analysis, Recommendation & Roadmap

Day 16 — Consolidate POC data

- Fill the weighted scorecard with actual measured data. No opinions without a metric behind them.
- Compute per-tool: avg. automation time per scenario → extrapolate effort for the full automatable backlog.

Day 17 — Coverage & ROI model

- Final coverage statement: e.g., "1,280 of 1,500 scenarios (85%) automatable; 150 need app/testability changes; 70 remain manual (hardware/exploratory)."
- Build ROI model: manual execution cost per regression cycle vs. automated, break-even timeline, 3-year TCO per tool.

Day 18 — Recommendation & roadmap

- Final recommendation (single tool or hybrid stack) with justification traced to criteria and POC evidence.
- Adoption roadmap: framework build-out, pilot apps, team training/hiring, phased scenario migration (quick wins first), governance (coding standards, review gates, flaky-test policy), 6/12/18-month milestones.

Day 19 — Report & presentation prep

- Write the proof-of-architecture report: estate analysis, evaluation framework, POC results with metrics, risks, recommendation, roadmap, cost model.
- Prepare an executive deck (10–12 slides) and a technical appendix.
- Internal dry run / peer review.

Day 20 — Presentation & handover

- Present to stakeholders (exec summary first, evidence available on demand).
- Hand over POC code, scorecards, and raw data.
- Agree next steps: procurement, pilot team, framework project kickoff.

Deliverables checklist

1. Application landscape & technology matrix
2. Test scenario classification with automatability tagging
3. Weighted evaluation framework (client-approved)
4. POC scripts + measured scorecard for each tool
5. Coverage feasibility statement (fact-based %, with the non-automatable list)
6. 3-year TCO & ROI model
7. Recommendation report + executive presentation
8. Adoption roadmap

Key risks to manage during the 4 weeks

- **Access delays** (licenses, environments) — raise on Day 1, escalate by Day 3.
- **Cherry-picked POC scenarios** — insist on including the hardest app types.
- **"100% automation" expectation** — reset with data in the Week 2 checkpoint, not on Day 20.
- **Tool bias from stakeholders** — the pre-agreed weighted scorecard is your defense.